4.2.5 When are Objects Saved?

When you assign an object to a `has_one` association, that object is automatically saved (in order to update its foreign key). In addition, any object being replaced is also automatically saved, because its foreign key will change too.

If either of these saves fails due to validation errors, then the assignment statement returns `false`and the assignment itself is cancelled.

If the parent object (the one declaring the `has_one` association) is unsaved (that is, `new_record?`returns `true`) then the child objects are not saved. They will automatically when the parent object is saved.

If you want to assign an object to a `has_one` association without saving the object, use the `association.build` method.

4.3 has_many Association Reference

The `has_many` association creates a one-to-many relationship with another model. In database terms, this association says that the other class will have a foreign key that refers to instances of this class.

4.3.1 Methods Added by has_many

When you declare a `has_many` association, the declaring class automatically gains 16 methods related to the association:

- `collection(force_reload = false)`
- `collection<<(object, ...)`
- `collection.delete(object, ...)`
- `collection.destroy(object, ...)`
- `collection=(objects)`
- `collection_singular_ids`
- `collection_singular_ids=(ids)`
- `collection.clear`
- `collection.empty?`
- `collection.size`
- `collection.find(...)`
- `collection.where(...)`
- `collection.exists?(...)`
- `collection.build(attributes = {}, ...)`
- `collection.create(attributes = {})`
- `collection.create!(attributes = {})`

In all of these methods, `collection` is replaced with the symbol passed as the first argument to `has_many`, and `collection_singular` is replaced with the singularized version of that symbol. For example, given the declaration:

```
class Customer < ActiveRecord::Base
  has_many :orders
end
```

Each instance of the `Customer` model will have these methods:

```
orders(force_reload = false)
orders<<(object, ...)
orders.delete(object, ...)
orders.destroy(object, ...)
orders=(objects)
order_ids
order_ids=(ids)
orders.clear
orders.empty?
```